

MXAI Detector

Documentacion tecnica del programa mobile de analisis Android con XAI

Elemento	Descripcion
Proyecto	Mobile X AI Detector
Plataforma	Flutter + Android nativo + Ollama + backend Python opcional
Modelo ML	Android_Malwaredetector/android_malware_detector.joblib
Fecha	22 de mayo de 2026

Resumen

MXAI Detector es una app Flutter orientada a analisis de comportamiento Android. La interfaz ejecuta un analisis global del dispositivo, recibe APKs externos por Android intents, muestra una decision Benign/Malware y usa Ollama para transformar probabilidades y factores locales en explicaciones legibles.

1. Proposito del programa

El programa combina deteccion de malware Android con explicabilidad. El usuario pulsa Analizar y el sistema observa apps, permisos y senales del dispositivo para producir un resultado global. Para APKs descargados, el flujo se activa fuera de la app mediante Compartir/Enviar a MXAI despues de la descarga.

La meta funcional final es conectar un extractor compatible con el modelo entrenado, producir un vector de 470 columnas y ejecutar android_malware_detector.joblib. En el estado actual, Flutter usa un motor prototipo calibrado y deja preparada una API Python para inferencia real con joblib.

2. Flujo conceptual de analisis

```
APK o comportamiento observado
-> Extractor de features compatible
-> Vector de 470 columnas
-> android_malware_detector.joblib
-> Prediccion Benign/Malware
-> SHAP / importancia local
-> Ollama explica el resultado
```

El punto central del flujo es la compatibilidad con las 470 columnas del modelo. La capa visual no decide por si sola: presenta la salida del motor de deteccion y separa la explicacion en dos funciones: reporte XAI del analisis global y chatbot defensivo de ciberseguridad.

3. Arquitectura general

Elemento	Descripcion
Flutter UI	Pantallas, scanner circular, navbar lateral, reporte XAI, chatbot y ventana externa de APK.
Android nativo	MethodChannel para listar apps instaladas, abrir ajustes de uso y recibir APKs compartidos.
Motor prototipo	PrototypeDetectionEngine genera una decision temporal mientras se conecta el extractor real.
Modelo ML	Metadata, features e importancia global se cargan como assets; el joblib queda para backend nativo/Python.
Ollama	mxai-xai-report genera reportes; mxai-cyber-chat responde preguntas defensivas; mxai-tinyllama-local valida el GGUF.
Backend Python	FastAPI recibe vector o diccionario de features, ejecuta joblib y devuelve probabilidades e importancia local.

4. Estructura de archivos principal

Elemento	Descripcion
lib/main.dart	Interfaz principal, pantallas, navegacion lateral y paneles de analisis.
lib/models/detector_model.dart	Modelos de datos: metadata, target, resultado y factores locales.
lib/services/detector_service.dart	Motor temporal de deteccion y calibracion conservadora.
lib/services/android_app_scanner_service.dart	Cliente Flutter del MethodChannel Android.
lib/services/ollama_service.dart	Cliente HTTP hacia /api/generate de Ollama.
backend/inference_api.py	Servidor FastAPI para ejecutar el modelo joblib con un vector compatible.
Android_Malwaredetector /	Modelo entrenado, features, metricas, mapping de clases e importancias.
ollama/	Modelfiles para reporte, chatbot y validacion del GGUF local.
scripts/	Automatizacion para crear modelos Ollama e instalar el APK por ADB.

5. Interfaz movil

La app usa una estetica futurista oscura, con fondo plano de baja interferencia, scanner circular y paneles compactos. La navbar vertical se abre al tocar el escudo y desplaza ligeramente el contenido para mantener orientacion espacial.

- Inicio: analisis global, scanner circular, boton Analizar y reporte XAI.
- Decision: explica como decide el sistema y muestra el resultado actual.
- Datos: resume senales observadas, features y factores locales.
- Chat XAI: chatbot limitado a ciberseguridad defensiva y preguntas sobre MXAI.
- SDK Android: permisos, servicios y flujo externo para APK descargado.

6. Analisis global

El analisis global evita falsos positivos por volumen bruto de permisos. Android y los fabricantes preinstalan muchas apps con permisos sensibles; por eso el motor prototipo separa apps de usuario y apps de sistema/OEM, mide concentracion de senales y exige un umbral mas alto antes de marcar Malware.

- Si no hay senales fuertes, el riesgo global queda en rango Benign.
- Si varias apps de usuario concentran permisos y senales de alto riesgo, el resultado puede elevarse.
- El reporte XAI aparece en Inicio y usa solo los datos del analisis generado.

7. Analisis de APKs descargados

MXAI no compite con la descarga ni con el instalador del sistema. El manifest no declara ACTION_VIEW para APK; declara ACTION_SEND. Eso fuerza el flujo correcto: primero el usuario descarga el APK, despues lo comparte o envia a MXAI para analizarlo.

```
Descarga del APK
-> Compartir / Enviar a MXAI
-> Ventana externa de analisis
-> Decision + explicabilidad
```

8. Modelo ML y backend de inferencia

android_malware_detector.joblib es un modelo scikit-learn serializado. Dart/Flutter no puede ejecutarlo directamente de forma nativa. Por eso el proyecto mantiene dos caminos: un motor prototipo para la interfaz movil y una API Python para inferencia real cuando exista el vector de 470 features.

```
python -m pip install -r backend\requirements.txt
python -m uvicorn backend.inference_api:app --reload --host 127.0.0.1 --port 8000
```

El endpoint POST /predict acepta vector con 470 valores o un diccionario de features. Devuelve prediccion, probabilidades Benign/Malware e importancia local. Si SHAP esta disponible, usa TreeExplainer; si no, calcula una aproximacion con feature_importances_.

9. Ollama y XAI

La app llama a Ollama mediante HTTP en /api/generate. En telefono fisico, 127.0.0.1 apunta al telefono, no a la PC. Por eso el APK se compila con OLLAMA_BASE_URL apuntando a la IP local del equipo donde corre Ollama.

Elemento	Descripcion
mxai-xai-report	Genera reporte del analisis global usando prediccion, probabilidades y factores locales.
mxai-cyber-chat	Responde preguntas defensivas de ciberseguridad y sobre la app MXAI.
mxai-tinyllama-local	Carga tinyllama.Q4_K_M.gguf como validacion de archivo local; no es el modelo activo si su salida es corrupta.

```
.\scripts\create_ollama_models.ps1
$env:OLLAMA_HOST="0.0.0.0:11434"
ollama serve
.\scripts\install_to_phone.ps1 -OllamaBaseUrl "http://TU_IP_LOCAL:11434"
```

Nota practica sobre Ollama

El APK no instala Ollama dentro de Android. Ollama corre como servidor externo en la PC o en otro host. Para una version 100% offline dentro del telefono, el siguiente paso seria integrar un runtime nativo como llama.cpp, ONNX Runtime o TFLite y cargar el modelo como asset compatible.

10. Permisos y servicios Android

Elemento	Descripcion
INTERNET	Permite conectar con Ollama o con un backend de inferencia en la red local.
QUERY_ALL_PACKAGES	Permite listar apps instaladas durante pruebas locales.
PACKAGE_USAGE_STATS	Prepara acceso a estadísticas de uso; requiere activación manual por el usuario.
ACTION_SEND APK	Permite recibir APKs después de descargarlos y compartirlos con MXAI.
AppAnalysisService	Servicio reservado para análisis en segundo plano cuando se conecte el extractor real.

11. Instalación y prueba

- 1 Conectar el teléfono por USB y activar depuración USB.
- 2 Crear los modelos Ollama con `scripts\create_ollama_models.ps1`.
- 3 Iniciar Ollama escuchando en red local con `OLLAMA_HOST=0.0.0.0:11434`.
- 4 Instalar con `scripts\install_to_phone.ps1`; el script detecta la IP local si no se pasa una manual.
- 5 Abrir la app, ejecutar Analizar y después generar el reporte XAI.

```
flutter analyze
flutter test
flutter build web
flutter build apk --debug
```

12. Estado actual y limitaciones

- La interfaz móvil, navbar lateral, scanner, reporte XAI y chatbot ya están implementados.
- El motor de Flutter sigue siendo prototipo hasta conectar el extractor de 470 columnas.
- El joblib se ejecuta desde backend Python o debería convertirse a ONNX/TFLite/plugin nativo.
- `tinylama.Q4_K_M.gguf` se importa en Ollama, pero debe reemplazarse por una variante `instruct/chat` si devuelve salida corrupta.
- La reducción de falsos positivos se basa en calibración conservadora y separación `user/system` apps.

13. Roadmap recomendado

- 1 Construir extractor real que emita exactamente las 470 features esperadas.
- 2 Conectar Flutter al backend `/predict` o portar el modelo a un formato móvil.

-
- 3 Agregar SHAP local real en la respuesta de inferencia y mapearlo al panel de Datos.
 - 4 Empaquetar una opcion offline de XAI con runtime nativo si se requiere funcionamiento sin PC.
 - 5 Endurecer permisos, privacidad y comunicacion de red antes de distribucion fuera de laboratorio.

14. Glosario breve

Elemento	Descripcion
APK	Paquete instalable de Android.
Feature	Variable numerica o categorica usada por el modelo.
Vector 470	Entrada ordenada que el modelo entrenado espera recibir.
SHAP	Tecnica de explicabilidad que estima contribuciones locales de features.
LLM	Modelo de lenguaje usado para redactar explicaciones a partir de datos tecnicos.
Ollama	Servidor local que ejecuta modelos de lenguaje y expone /api/generate.